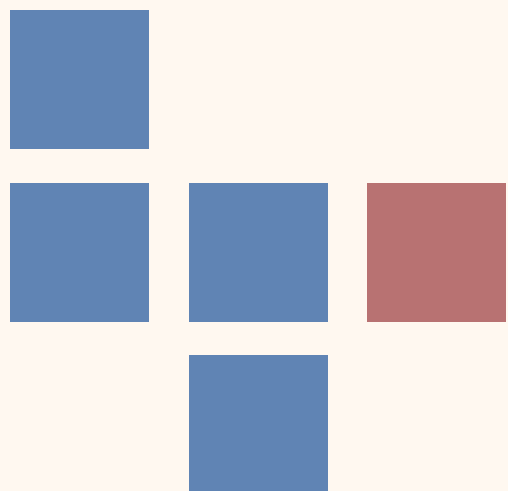




ATIVIDADE 1 - SISTEMAS OPERACIONAIS (2026.2)

ALUNA: MARIANA SILVA COSTA

TRILHA: TRILHA B - RAG TEXTUAL (OLLAMA-LOCAL-RAG)

MODELO: META-LLAMA-3.1-8B-INSTRUCT

Problema

Analisar como modelos de IA generativa utilizam recursos do sistema operacional quando executadas localmente, especialmente como a concorrência de requisições e o tamanho da base de conhecimento influenciam o tempo de resposta, o uso de CPU e RAM, além da quantidade de processos e threads. Neste trabalho, foi utilizado um sistema RAG textual local com Ollama, utilizando o modelo Meta-Llama-3.1-8B-Instruct.

Arquitetura do Sistema

- Camada de Aplicação: Python / Langchain rodando via uvicorn.
- Motor de Inferência: API Local Ollama (porta 127.0.0.1:11434).
- Gerenciamento do SO: Serviços coordenados pelo systemd do Linux

Ambiente

SO: Fedora Linux 44 (Instalação nativa).

Kernel: Linux 7.1.13-200.f.c44.x86_64

CPU: AMD Ryzen 5 5600GT com gráficos Radeon integrados.

Núcleos / Threads: 6 núcleos físicos / 12 threads

RAM: 16 GB.

Tipo de ambiente: Nativo.

Memória: 256 GB.

Versão do Ollama: 0.34.0

Justificativa do modelo

8B parâmetros: dentro do limite de 10B.

Contexto amplo: indicado para documentos e RAG.

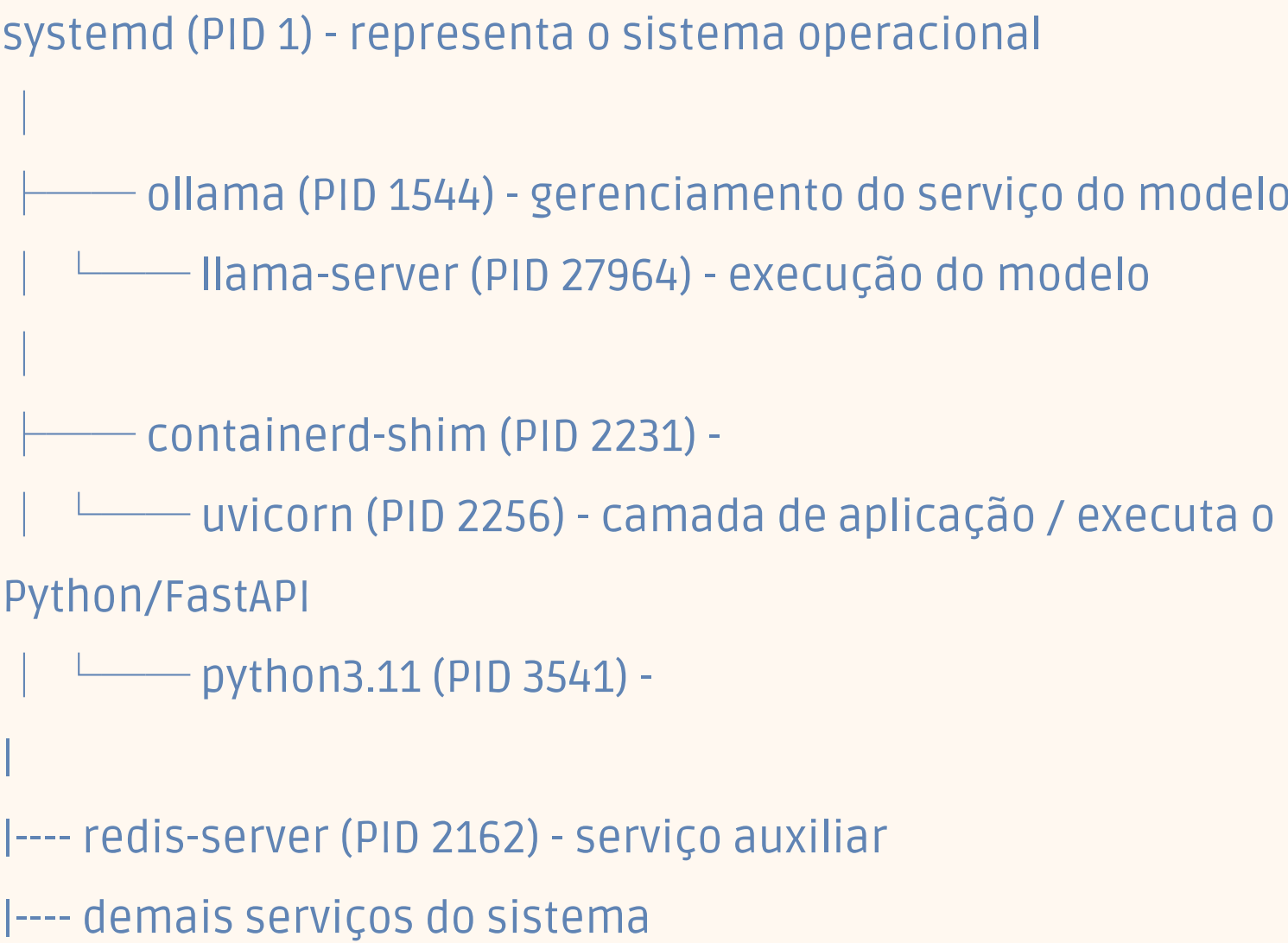
Execução local: utilizando o Ollama.

GGUF + Q4_K_M: reduz o uso de memória e processamento.

Bom equilíbrio: mantém qualidade nas respostas com menor custo computacional.

Métricas: facilita a análise de CPU, RAM, processos e threads.

Processos e Hierarquia



COMM	llama-ser ver	ps	uvicor n	python 3.11	redis-ser ver	container d-shim	ollama
PID (id do processo)	27964	28128	2256	3541	2162	2231	1544
PPID (id do processo pai)	1544	28037	2231	2256	2138	1	1
STAT (estado)	Sl (em espera e multithrea d)	Sl (em espera e multithr ead)	Ssl (líder, em esper a e multit hread)	Sl (em espera e multithr ead)	Ssl (líder, em espera e multithre ad)	Sl (em espera e multithrea d)	Ssl (líder, em espera e multithrea d)
%CPU	60.1	50.0	0.0	0.1	0.3	0.0	0.0
%MEM	34.3	0.0	0.1	0.5	0.0	0.0	0.3
NLWP (Threads)	21	1	2	5	6	11	19

Syscalls e I/O

Fase de Indexação (Leitura de Dados)

- Predomínio de chamadas de arquivo para ler os .md: newfstatat, openat e read.

Fase de Inferência (Consultas)

- Predomínio de comunicação via sockets de rede local: accept4, sendto, recvfrom e poll.

O gargalo de Sincronização

- Em ambas as fases, a chamada futex foi responsável pela maior parte do tempo, evidenciando o trabalho do SO na sincronização e espera entre as threads.

Metodologia

Foram executadas baterias com perguntas curtas e longas, divididas em 4 cenários:

- **C1 (Padrão):** Bases de dados originais, 1 único terminal.
- **C2 (Concorrência):** Bases de dados originais, 2 terminais rodando simultaneamente.
- **C3 (Base Maior):** Adição de 5 novos arquivos (Redes, Sistemas Operacionais, IA, etc.), 1 único terminal.
- **C4 (Base Maior + Concorrência):** As 5 novas bases consultadas em 2 terminais simultâneos.

Resultados

Responsividade (Latência)

- O tempo de resposta para um prompt longo disparou de 2m45s (em C1) para impressionantes 5m27s (em C2, terminal 1).

Competição por Recursos

- A CPU tornou-se o principal gargalo do sistema, trabalhando rotineiramente acima de 100% de uso sob concorrência.
- O uso de RAM manteve estabilidade técnica na casa dos 34% a 36%.

Resultados

Falha de Recuperação (Alucinação)

- Com a introdução das 5 bases aleatórias (C3 e C4), o modelo perdeu a assertividade em prompts longos.
- O sistema RAG justificou não achar a resposta focando em documentos errados (Sistemas de Arquivos e Redes) e ignorando a documentação alvo.

Conclusão Operacional

- Aumentar o contexto às cegas confunde a busca semântica, forçando o LLM a abortar a instrução primária.

Limitações

Arquitetura Unificada de Memória:

- A ausência de uma GPU dedicada (uso da Radeon integrada) força o compartilhamento físico do barramento de RAM entre o processamento gráfico e a CPU, dificultando a medição isolada de VRAM.

Desvio de Comportamento:

- As falhas de contexto (alucinação) nos cenários C3 e C4 interromperam o fluxo do LLM precocemente, impedindo a medição contínua de sobrecarga pura de hardware nessas etapas.

Uso Crítico de IA Generativa

Aplicações Bem-Sucedidas:

- Uso para gerar as 5 bases de dados .md sintéticas utilizadas nos testes C3 e C4, além da geração de ideias para prompts e tabelas.

Rejeição e Senso Crítico (Correção Manual):

- A IA sugeriu códigos obsoletos de importação da biblioteca langchain.chains.
- As sugestões quebraram o sistema, obrigando a refatoração manual dos scripts `create_database.py` e `query_data.py` para os padrões recentes (LCEL) e a instalação isolada do `faiss-cpu`.